

ALGORITMI DI ORDINAMENTO

Prof.ssa Taffurelli



INTRODUZIONE

- Gli algoritmi di ordinamento (sorting algorithms) sono procedure che riorganizzano una collezione di dati secondo un certo criterio, di solito crescente o decrescente.
- Sono fondamentali in informatica perché:
 - rendono più efficienti ricerche e analisi dei dati
 - sono alla base di molte strutture dati e algoritmi complessi
 - permettono di confrontare approcci diversi alla risoluzione di un problemi



Algoritmi semplici (didattici)

Sono facili da capire e implementare, ma poco efficienti su grandi quantità di dati.

Algoritmo	Idea di base	Complessità media
Bubble Sort	Confronta coppie adiacenti e "fa salire" i valori maggiori	$O(n^2)$
Selection Sort	Seleziona ogni volta il minimo e lo mette al posto giusto	$O(n^2)$
Insertion Sort	Inserisce ogni elemento nella posizione corretta	$O(n^2)$

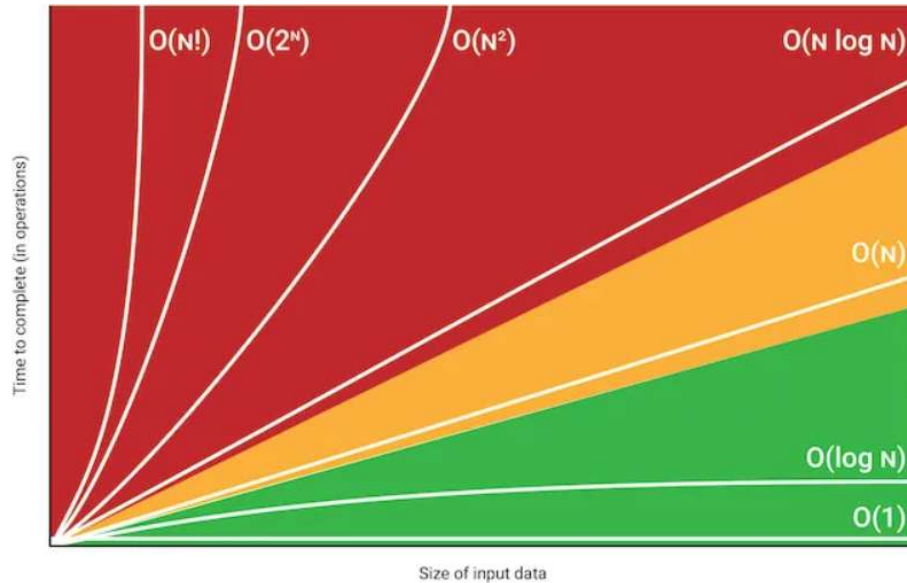
Algoritmi efficienti

Usano strategie più intelligenti per ridurre il numero di confronti.

Algoritmo	Paradigma	Complessità media
Merge Sort	Divide et impera	$O(n \log n)$
Quick Sort	Divide et impera	$O(n \log n)$ (ma può degradare a $O(n^2)$)
Heap Sort	Uso di una struttura heap	$O(n \log n)$



Cos'è la complessità algoritmica?



Cos'è la complessità algoritmica?

- Misura quanto tempo e quanta memoria un algoritmo richiede in funzione della dimensione dell'input.
- Fondamentale per garantire scalabilità e prestazioni in sistemi con grandi volumi di dati.

Cos'è Big O

- È una notazione matematica che descrive il comportamento asintotico di un algoritmo.
- Permette di analizzare tempo di esecuzione e uso di memoria al crescere dei dati

Cos'è la complessità algoritmica?

Notazione	Nome	Descrizione
$O(1)$	Costante	Tempo fisso, indipendente dalla dimensione dell'input
$O(\log n)$	Logaritmica	Crescita lenta; tipico della ricerca binaria
$O(n)$	Lineare	Cresce proporzionalmente all'input
$O(n \log n)$	Log-lineare	Tipico degli algoritmi di ordinamento efficienti (es. Merge Sort)
$O(n^2)$	Quadratica	Crescita rapida; frequente in algoritmi con doppio ciclo annidato
$O(2^n)$, $O(n!)$	Esponenziale/Fattoriale	Crescita estrema; algoritmi ricorsivi o combinatori

CLASSIFICAZIONE

- La **prima classificazione** può essere effettuata in base alla **complessità di calcolo**.
 - 1. Algoritmi Semplici di Ordinamento.** Algoritmi che presentano una complessità proporzionale a n^2 , dove n è il numero di informazioni da ordinare. Essi sono generalmente caratterizzati da poche e semplici istruzioni:
 - Selection sort
 - Insertion Sort
 - Bubble sort
 - 2. Algoritmi Evoluti di Ordinamento.** Algoritmi che offrono una complessità computazionale proporzionale a $n \cdot \log_2 n$ (che è sempre inferiore a n^2). Tali algoritmi sono molto più complessi e fanno molto spesso uso di ricorsione. La convenienza del loro utilizzo si ha unicamente quando il numero n di informazioni da ordinare è molto elevato
 - Quick sort
 - Merge sort

2. algoritmi semplici di ordinamento: Selection sort

```
static void SelectionSort(int[] array){
    int n = array.Length;

    for (int i = 0; i < n - 1; i++){
        // Trova l'indice del valore minimo nel sottovettore non ordinato
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (array[j] < array[minIndex])
                minIndex = j;
        }

        // Scambia il valore min trovato con il 1° elemento non ordinato
        if (minIndex != i)
        {
            int temp = array[i];
            array[i] = array[minIndex];
            array[minIndex] = temp;
        }
    }
}
```

- Il selection sort scorre l'array trovando ogni volta l'elemento minimo (o massimo) e spostandolo nella posizione corretta.
- È un algoritmo semplice, ma ha una complessità $O(n^2)$, quindi non è adatto ad array molto grandi.

https://www.youtube.com/watch?v=JWfNkvQ_-fE

2. algoritmi semplici di ordinamento: Insertion sort

```
static void InsertionSort(int[] array)
{
    int n = array.Length;

    for (int i = 1; i < n; i++)
    {
        int key = array[i];
        int j = i - 1;

        // Sposta gli elementi dell'array[0..i-1] che sono maggiori di key
        // di una posizione in avanti
        while (j >= 0 && array[j] > key)
        {
            array[j + 1] = array[j];
            j--;
        }

        array[j + 1] = key;
    }
}
```

- Parte dal secondo elemento e lo confronta con quelli a sinistra.
- Sposta a destra gli elementi più grandi per inserire il nuovo elemento nella posizione corretta.
- È molto efficiente per array quasi ordinati.
- Complessità $O(n^2)$ nel caso peggiore (grandi quantità di dati disordinati)

<https://www.youtube.com/watch?v=DHtJHDIhh5c>

2. algoritmi semplici di ordinamento: bubble sort

```
void scambia(ref int a, ref int b){
    int z;
    z = a;
    a = b;
    b = z;
}

void BubbleSort(int[] v, int dim){
    bool ordinato=false;
    int i,j;
    for (j=0; j<dim-1 && !ordinato; j++) {
        ordinato=true;
        for (i=dim-1; i>j; i--)
            if (v[i]<v[i-1]) {
                scambia(ref v[i],ref v[i-1]);
                ordinato=false;
            }
    }
}
```

- Versione ottimizzata del BubbleSort, che esce prima dal ciclo se l'array è già ordinato (ordinato=true)
- Non è un algoritmo efficiente perché effettua circa $N^2/2$ confronti ed $N^2/2$ scambi sia in media che nel caso peggiore. Il tempo di esecuzione dell'algoritmo è $O(N^2)$

<https://www.youtube.com/watch?v=KYmsWSk0rUU>